Table of Contents

Contents

Task Decomposition with Abstract Code: 3
- Login 3
- Register 3
- Main menu 3
- List Item to Sell 4
- Item Search Query 5
- View Description & Bids & Edit Description 9
- View Ratings and Delete Ratings 11
- Auction Results 12
- Item Results & Winners 13
- Submit Ratings 14
- Category Report 15
- User Report 15
- Top Rated Items Report 16
- Auction Statistics 17
- Canceled Auction Details 18

Task Decomposition with Abstract Code:

Login

Abstract Code

- User enters username ('$Username), password ('$Password') input fields.
- If data validation is successful for both username and password input fields, then:
    - When ***Enter*** button is clicked:

```
SELECT password FROM User WHERE user_name='$Username';
```

        - If User record is found but user.password != '$Password':
            - Go back to the **Login** form, with an error message.
        - Else:
            - Store login information as session variable '$Username.
            - Go to **Main Menu** form.
- Else email and password input fields are invalid, display **Login** form, with error message.

Register

Abstract Code

- User must fill the fields. ($*FirstName*, $*LastName*, $*Username*, $*Password*, *Confirm Password*)
- When user clicks on ***Register*** button.
    - If all the fields are not filled:
        - Mention an error msg/ warning that all the fields are required.
    - If Username is already being used:
        - Mention an error msg/ warning that the username is already taken.
    - If the password and confirm password don't match:
        - Mention an error msg/ warning that they should match
    - If all other subtasks are successful, create a new record in the User entity table

```
INSERT INTO `User` (user_name, password, first_name, last_name)
VALUES ($UserName, $Password, $FirstName, $LastName);
```

Main menu

Abstract Code

- After a user logs in, the firstname and lastname should be listed on the main menu.
- Based on the user_type:
    - If the user is not a administrative user:
        - Show only "**Search for Items**", "**List an Item for Sale**", "**View Auction**" tabs.

    - Else:
        - Along with "**Search for Items**", "**List an Item for Sale**", "**View Auction**" tabs. Display report tabs "**Category Report**", "**User Report**", "**Top Rated Items**", "**Auction Statistics**", "**Cancelled Auction Details**".
- Upon:
    - Click ***Search for Items*** button - Jump to the **Item Search Query** task.
    - Click ***List an Item for Sale*** button - Jump to the **List Item to Sell** task.
    - Click ***View Auction*** button - Jump to the **Auction Results** task.
    - Click ***Category Report*** button - Jump to the **Category Report** task.
    - Click ***User Report*** button - Jump to the **User Report** task.
    - Click ***Top Rated Items*** button - Jump to the **Top Rated Items** task.
    - Click ***Auction Statistics*** button - Jump to the **Auction Statistics** task.
    - Click ***Cancelled Auction Details*** button - Jump to the **Cancelled Auction Details** task.

List Item to Sell

Abstract Code

- Read the database records from the Category entity
    - Display *Category* field dropdown values

```
SELECT category_name FROM Category;
```

- User inputs values for fields $*ItemName, $Description, $Category, $Condition, $StartingBid, $MinSalePrice, $AuctionLength, $GetItNowPrice, $Returnable*
- Check for business constraint $StartingBid < $MinSalePrice < $GetItNowPrice:
    - If true: Create new records for Item and Auction entity

```
-- application gets UserName of current user as $UserName

INSERT INTO Item (listed_by, item_name, description, returnable, category,
item_condition)

VALUES ($UserName, $ItemName, $Description, $Returnable, $Category, $Condition);
```

```
-- application calculates $ScheduledEndTime from $AuctionLength and current date time

--itemID generated for the inserted record in Item table is fetched in $itemID

INSERT INTO Auction (item_ID, starting_bid, min_sale_price, getit_now_price, auction_length, scheduled_end_time)

VALUES ($itemID, $StartingBid, $MinSalePrice, $GetItNowPrice, $AuctionLength, $ScheduledEndTime);
```

- If false: Display error message to correct input values

Item Search Query

Abstract Code

- User enters search criteria in the **Item Search** form and hits ***Search*** button
- If Keyword is present, filter items whose item_name or description contains the keywords in anywhere and proceed to the next step. If no keyword is present i.e empty, select all the items and proceed to the next step.

  ItemFilter1:

```
SELECT item_ID FROM Item WHERE
        CASE WHEN $Keyword IS NOT NULL
        THEN
        description LIKE '%{ $Keyword}%' OR item_name LIKE '%{$Keyword}%'
        ELSE 1=1
        END;
```

- If Category is selected from dropdown, filter items whose Category matches the selection and proceed to the next step. If no selection is made in the dropdown, select all the items and proceed to the next step.

  ItemFilter2:

```
SELECT item_ID FROM Item WHERE
   CASE WHEN $Category IS NOT NULL
   THEN
   category LIKE '%$Category%'
   ELSE 1=1
   END;
```

- If Minimum price is present,
  - if there are bids on the item
    - calculate max bid for the item

        - filter items whose max_bid is greater than or equal to minimum price in search criteria

else, filter items whose starting_price is greater than or equal to minimum price in search criteria

_ItemFilter3:

```
WITH MaxBid(item_ID,max_bid) AS
(SELECT item_ID, MAX(bid_amount) max_bid FROM ItemBid WHERE item_ID IN
(SELECT item_ID FROM ItemFilter2) GROUP BY item_ID)
SELECT i.item_ID FROM Item i
INNER JOIN Auction a ON a.item_ID = i.item_ID
LEFT JOIN MaxBid m ON i.item_ID=m.item_ID
WHERE i.item_ID IN (SELECT item_ID FROM ItemFilter2)
 AND COALESCE(m.max_bid, a.min_sale_price) >= COALESCE( $MinPrice,0)
```

- If Maximum price is present,
    - if there are bids on the item
        - skip items whose starting_price is greater than the maximum price in search criteria
        - calculate max bid for the item
        - filter items whose max_bid is less than or equal to maximum price in search criteria

else, filter items whose starting_price is less than or equal to minimum price in search criteria

_ItemFilter4:

```
WITH MaxBid(item_ID,max_bid) AS
(SELECT item_ID, MAX(bid_amount) max_bid FROM ItemBid
WHERE item_ID IN (SELECT item_ID FROM ItemFilter3) GROUP BY item_ID)
SELECT i.item_ID FROM Item i
INNER JOIN Auction a ON a.item_ID = i.item_ID
LEFT JOIN MaxBid m ON i.item_ID=m.item_ID
WHERE  i.item_ID IN (SELECT item_ID FROM ItemFilter3) AND
CASE WHEN $MaxPrice IS NOT NULL
THEN
a.min_sale_price <= $MaxPrice AND COALESCE(m.max_bid, a.min_sale_price) <=
$MaxPrice
ELSE 1=1 END
```

- If Condition is selected from the dropdown, filter items whose Condition matches the selection or better than the selection(New>Very good>Good>Fair>Poor). If no selection is made in the dropdown, select all the items and return.

_ItemFilter5:

```
SELECT item_ID FROM Item WHERE
item_ID IN (SELECT item_ID FROM ItemFilter4) AND
CASE
```

```
            WHEN $Condition='New' THEN item_condition IN ('New')
            WHEN  $Condition='Very Good' THEN item_condition IN ('New','Very Good')
             WHEN $Condition='Good' THEN item_condition IN ('New','Very Good','Good')
           WHEN $Condition='Fair' THEN item_condition IN ('New','Very
Good','Good','Fair')
            ELSE item_condition IN ('New','Very Good','Good','Fair','Poor')
 END;
```

- The **Search Results** returned should contain the item_ID, item_name, current_bid, user_name of the highest bidder, the item's get_it_now_price and the auction_end_time of the item.
- The items should be displayed based on the Auction end date with the auction ending the soonest listed first.
- when the user clicks on the item name, it should take to the item description page which is described below in the "**View Description & Bids**" task.

```
WITH ItemFilter1(item_ID) AS

( SELECT item_ID FROM Item WHERE

                CASE WHEN $Keyword IS NOT NULL
                THEN
                description LIKE '%{ $Keyword}%' OR item_name LIKE '%{$Keyword}%'
                ELSE 1=1
END),
ItemFilter2(item_ID) AS

(
 SELECT item_ID FROM Item WHERE
          item_ID IN (SELECT item_ID FROM ItemFilter1) AND

          CASE WHEN $Category IS NOT NULL
           THEN
           category LIKE '%$Category%'
           ELSE 1=1
          END
),

ItemFilter3(item_ID) AS

(WITH MaxBid(item_ID,max_bid) AS

                (SELECT item_ID, MAX(bid_amount) max_bid FROM ItemBid WHERE item_ID IN (SELECT
                item_ID FROM ItemFilter2) GROUP BY item_ID)
                SELECT i.item_ID FROM Item i
                INNER JOIN Auction a ON a.item_ID = i.item_ID
                LEFT JOIN MaxBid m ON i.item_ID=m.item_ID
                WHERE i.item_ID IN (SELECT item_ID FROM ItemFilter2)
 AND COALESCE(m.max_bid, a.min_sale_price) >= COALESCE( $MinPrice,0)),
```

```
ItemFilter4(item_ID) AS

(WITH MaxBid(item_ID,max_bid) AS
(SELECT item_ID, MAX(bid_amount) max_bid FROM ItemBid
WHERE item_ID IN (SELECT item_ID FROM ItemFilter3) GROUP BY item_ID)
SELECT i.item_ID FROM Item i
INNER JOIN Auction a ON a.item_ID = i.item_ID
LEFT JOIN MaxBid m ON i.item_ID=m.item_ID
WHERE  i.item_ID IN (SELECT item_ID FROM ItemFilter3) AND

        CASE WHEN $MaxPrice IS NOT NULL
THEN
a.min_sale_price <= $MaxPrice AND COALESCE(m.max_bid, a.min_sale_price) <= $MaxPrice
ELSE 1=1 END),

ItemFilter5(item_ID) AS

(SELECT item_ID FROM Item WHERE
item_ID IN (SELECT item_ID FROM ItemFilter4) AND

        CASE
                WHEN $Condition='New' THEN item_condition IN ('New') WHEN  $Condition='Very Good'
        THEN item_condition IN ('New','Very Good')
                WHEN $Condition='Good' THEN item_condition IN ('New','Very Good','Good')
               WHEN $Condition='Fair' THEN item_condition IN ('New','Very Good','Good','Fair')
                ELSE item_condition IN ('New','Very Good','Good','Fair','Poor')
         END
),ItemHighestBid AS

(SELECT item_ID, max(bid_amount) max_bid FROM ItemBid WHERE

item_ID IN (SELECT item_ID FROM ItemFilter5)

GROUP BY item_ID ORDER BY item_ID),

ItemFilter6(item_ID,current_bid,user_name) AS

(SELECT ib.item_ID, ib.bid_amount current_bid, ib.bid_by user_name FROM ItemBid ib INNER JOIN
ItemHighestBid ihb

ON ib.item_ID=ihb.item_ID
WHERE ib.item_ID IN (SELECT item_ID FROM ItemFilter5))

SELECT i.item_ID, i.item_name, a.getit_now_price,a.scheduled_end_time
auction_end_time,hb.current_bid,hb.user_name

 FROM Item i

INNER JOIN Auction a ON a.item_ID= i.item_ID

 LEFT JOIN ItemFilter6 hb ON i.item_ID = hb.item_ID

ORDER BY auction_end_time;
```

View Description & Bids & Edit Description

Abstract Code

- To view item description and Bids:
  - Get item details and add to result the following fields - item_ID, item_name, description, returnable
  - Get category and condition of the item and add it to result

    ```
    SELECT item_name, description, returnable, category, item_condition
    FROM Item WHERE item_id = $ItemId;
    ```

  - Get bid details such as bid_amount, bid_datetime, and username and add it to result

    ```
    SELECT bid_amount,time_of_bid,bid_by  FROM ItemBid WHERE
    item_id=$ItemId
    ```

  - Add current user's bid amount to your_bid and add it to result
  - Get get_it_now_price from Auction if present and add it to result

    ```
    SELECT getit_now_price FROM Auction  WHERE item_id=$ItemId
    ```

  - ***Edit description*** button should be enabled only for the user who listed the item .
  - ***Cancel item*** button should be enabled only for the admin users.
  - Return result
- To edit item description:
  - Upon clicking the ***edit description*** button, display a pop-up window to enter the new description. Thereby, the existing description is replaced with the new one.

    ```
    UPDATE Item SET description=$Description WHERE item_id=$ItemId
    ```

Get It Now & Enter Bid & Cancel Auction

Abstract Code

- To purchase the item via get it now:

- Fetch Item, Auction details for the item

```
SELECT i.item_ID, listed_by, item_name, description, returnable, item_condition, category
FROM Item i
LEFT JOIN Auction a ON a.item_ID = i.item_ID
WHERE i.item_ID=$item_ID;
```

- First verify that the user is not the one who listed this item. If the user hits ***get it now*** button
  - Check if the auction is active for the item.
  - The user purchases this item and the auction ends. Set the winner as the $user_name of the user who purchased the item. The ActualEndTime is then changed to this time, and the SalePrice is set to the get it now price.
  - Auction is updated

```
UPDATE Auction
SET sale_price=$get_it_now_price, winner = $user_name, actual_end_time = CURRENT_TIMESTAM
WHERE item_id=$Item_ID;
```

- To submit a bid on current item:
- Fetch Auction, ItemBid details for the item

```
SELECT a.item_ID,
        starting_bid,
        min_sale_price,
        getit_now_price,
        auction_length,
        scheduled_end_time,
        actual_end_time,
        sale_price,
        winner,
        canceled_time,
        canceled_by,
        cancelation_reason
FROM Auction a
LEFT JOIN ItemBid ib ON a.item_ID = ib.item_ID
WHERE a.item_ID=$item_ID
ORDER BY bid_amount, time_of_bid;
```

- The user enter a new bid $bid_amount for this item
- Conditions to submit a bid for an item: the item listing is still active; the $bid_amount >= max(bid_amount)+1 if there have been bids on this item or $bid_amount > min_sale_price if no bid has been entered for this item; $bid_amount < get_it_now_price
- The user clicks on ***Bid on this Item*** button
- Error message is displayed if the conditions are not met for the current bid.

- ItemBid is updated

```
INSERT INTO ItemBid (bid_by, item_id, bid_amount, time_of_bid)
VALUES ($user_name, $item_id, $bid_amount, CURRENT_TIMESTAMP);
```

- To cancel an auction:
- The admin can cancel the auction by themselves or by request from the regular user who listed the item. Verify that the current user is an admin user.
- After clicking on ***Cancel This Item***, The admin user will be prompted to provide the cancellation reason (which is optional) $cancellation_reason. The winner and the sale_price are left blank. The date and time of cancellation will be recorded as the actual_end_time and cancellation_time in Auction.
- Update Auction

```
UPDATE Auction
SET cancelation_reason = $cancellation_reason, actual_end_time =
CURRENT_TIMESTAMP, canceled_time = CURRENT_TIMESTAMP
WHERE item_ID=$item_ID;
```

View Ratings and Delete Ratings

Abstract Code

- The item_name and item_ID of the current item for which the user clicks ***ViewRatings*** are stored in $ItemName and $itemID
- Get all Items with the same item_name and their associated ItemRatings
    - Display *Stars, Comment, RateDateTime* for each ItemRating

```
-- item_name of the current item for which the user clicks ViewRatings is stored in
$ItemName

SELECT R.rated_by, R.rate_date_time, R.stars, R.rating_comment

FROM Item AS I, ItemRating AS R

WHERE I.item_name = $ItemName AND I.item_ID = R.item_ID
```

- The average of each ItemRating's stars is calculated

```
SELECT AVG(stars) FROM Item AS I, ItemRating AS R WHERE I.item_name =
$ItemName AND I.item_ID = R.item_ID
```

- Check if the current logged in user $UserName matches any of the rated_by in the previous query result.
    - If yes, Display ***Delete My Rating*** button next to that rating
- If User clicks on ***Delete My Rating*** button, remove the record from ItemRating

```
DELETE FROM ItemRating WHERE rated_by = $UserName AND item_ID =
$ItemID
```

- Refresh the screen to display the **ItemRatings** page again by running the **View Ratings** task again

Auction Results

Abstract Code

- If the ***Get it now*** button is hit, or the auction time has expired for an item, or the auction has been cancelled:
    - If the item was purchased using the ***Get It Now*** option
        - The user purchases this item and the auction ends. Set the winner as the $user_name of the user who purchased the item. The actual_end_time is then changed to this time, and the sale_price is set to the get it now price. Auction is then updated
    - If the auction has expired for an item:
        - Fetch ItemBid and get the highest $bid_amount and the $user_name of the person with the highest $bid_amount

```
WITH MAX_BID(item_ID, max_bid) AS
(SELECT item_ID, max(bid_amount)
FROM ItemBid
WHERE item_ID=$ItemID
GROUP BY item_ID)
SELECT bid_by, i.item_ID, max_bid
FROM ItemBid i
INNER JOIN MAX_BID m
ON i.item_ID = m.item_ID;
```

        - If the item has bids on it, set sale_price as the value of the highest $bid_amount and the winner as the $user_name of the person with the highest $bid_amount; set the actual_end_time as the $scheduled_end_time. Update Auction

```
UPDATE Auction
SET sale_price=$max_bid_amount, winner = $user_name, actual_end_time =
$scheduled_end_time
WHERE item_id=$item_ID;
```

        - If no one purchased the item using the ***Get It Now*** option, or there are no bids at all, both the sale_price field and the winner field will be left blank. Set the actual_end_time as the $scheduled_end_time. Update Auction

    - If the auction was cancelled:
        - sale_price and winner will be left blank; cancellation reason is

```
UPDATE Auction
SET actual_end_time = $scheduled_end_time
WHERE item_id=$item_ID;
```

recorded; the cancellation date/time is listed, and the actual_end_time will be set to cancellation time

- The above validation is done to determine the auction results when the ***View auction result*** button is clicked from the **View Main Menu** task.
- Fetch from Auction and display the auction results for recently ended items, sorted by auction actual_end_time (most recently ended auction first) with item_ID, item_name, sale_price, winner

```
SELECT i.item_ID, item_name, sale_price, winner, actual_end_time
FROM Auction a
INNER JOIN Item i
ON i.item_ID=a.item_ID
ORDER BY actual_end_time DESC
LIMIT 5;
```

Item Results & Winners

Abstract Code

- Get item details and add to result the following fields - *item_ID, item_name, description, returnable*
- Get *category* and *condition* of the item and add it to result
- Get *get_it_now_price* from Auction if present and add it to result
- Get *Auction ended* from Auction Results and add it to the result.

```
SELECT i.item_ID, item_name, description, returnable, category, item_condition,
a.getit_now_price ,a.scheduled_end_time AS AuctionEnded
FROM Item i INNER JOIN Auction a
        ON a.item_ID =i.item_ID and i.item_ID=$ItemId
```

- Get bid details such as *bid_amount, bid_datetime*, and *username* and add it to result
- Sort the bids by *Bid Amount* (descending)

- If the item has a winner then highlight the first bid row to green.
- If the item has no winner then highlight the first bid row to yellow.
- If the item auction is cancelled then highlight the first bid row to red and display the username as “Administrator”.

```
SELECT a.item_ID,
CASE WHEN a.canceled_time IS NULL THEN CONCAT('$', bid_amount) ELSE
'Cancelled' END AS bid_amount ,
COALESCE(a.canceled_time, time_of_bid) bid_datetime,
CASE WHEN a.canceled_time IS NULL THEN bid_by ELSE  'Administrator' END AS
Username
FROM Auction a LEFT JOIN ItemBid i
        on i.item_ID =a.item_ID AND a.item_ID=$ItemID
ORDER BY bid_amount DESC
```

- Return result

Submit Ratings

Abstract Code

- Check if the business constraint is met that current logged-in user $UserName is the winner of the item auction $ItemID

- Display the *My Rating (*Stars) and *Comments* on the **ItemRatings** page at the bottom.
- User enters input and clicks on the ***Rate This Item*** button, user input stored in $Stars and $Comments
- Create a new record for ItemRating entity and rates relationship with User and has relationship with Item

```
-- Current logged-in user_name stored in $UserName

-- Current item_ID stored in $ItemID

-- Form input stored in $Stars and $RatingComment

-- Current system date time stored in $CurrentDateTime

INSERT INTO ItemRating (rated_by, item_ID, stars, rating_comment,
rate_date_time)

VALUES ($UserName, $ItemID, $Stars, $RatingComment, $CurrentDateTime)
```

- UI screen should refresh by running the **View Ratings** task again so that the recently added rating is shown under in under the ***Delete My Rating*** section

Category Report

Abstract Code

- User clicked on ***Category Report*** button from **Main Menu**.
- Run the Category Report task; query the information about items to get an aggregate report based on the category.

```
SELECT
c.category_name,
COUNT(a.item_id) AS TotalItems,
MIN(CASE WHEN a.getit_now_price IS NOT NULL THEN a.getit_now_price END) AS
MinPrice,
MAX(CASE WHEN a.getit_now_price IS NOT NULL THEN a.getit_now_price END) AS
MaxPrice,
AVG(CASE WHEN a.getit_now_price IS NOT NULL THEN a.getit_now_price END) AS
AveragePrice
FROM Category c
LEFT JOIN Item i
        ON c.category_name = i.category
LEFT JOIN Auction a
          ON i.item_id = a.item_id
WHERE
          a.canceled_time is NULL
GROUP BY c.category_name
```

User Report

Abstract Code

- Read from ItemRating, User, Item, Auction
- Aggregate information about users:
  - Total number of items listed by the user (whether sold or not)
  - Total number of items sold by the user (either via Get It Now or auction)
  - Total number of items won by the user (either via Get It Now or auction)
  - Total number of items that the user rated
  - if the user has listings:
    - the most frequent item condition for the user's listings (in the event of a tie, the lowest-quality condition should be used).
  - If the user has no listings:
    - this field should display "N/A".
- Return report with UserName and aggregated information

```
SELECT
u.User_name AS UserName,
COUNT(DISTINCT i.item_ID) AS total_items_listed,
COUNT(DISTINCT CASE WHEN a.sale_price IS NOT NULL THEN a.item_ID END) AS total_items_sold,
COUNT(DISTINCT CASE WHEN a.winner IS NOT NULL THEN a.item_ID END) AS total_items_won,
COUNT(DISTINCT CASE WHEN ir.stars IS NOT NULL THEN ir.item_ID END) AS total_items_rated,
CASE
        WHEN COUNT(i.item_ID) = 0 THEN 'N/A'
        ELSE (SELECT item_condition
                                FROM (
                                                SELECT
                                                    listed_by,
                                                    item_condition,
                                                     ROW_NUMBER() OVER (PARTITION BY listed_by ORDER
            BY COUNT(*) DESC, item_condition) AS rn
                                                  FROM Item
                                                  WHERE listed_by = u.user_name
                                                   GROUP BY listed_by, item_condition
                                                ) AS ranked_conditions
                                WHERE rn = 1)
                                                     END AS most_frequent_condition
FROM User u
LEFT JOIN Item i ON u.user_name = i.listed_by
LEFT JOIN Auction a ON i.item_ID = a.item_ID
LEFT JOIN ItemRating ir ON u.user_name = ir.rated_by
GROUP BY u.user_name
ORDER BY u.user_name;
```

Top Rated Items Report

Abstract Code

- Read stars from ItemRating and item_name from Item entities.
- Calculate Average Rating of all the stars from ItemRatings related to each Item with same item_name
- Filter Top 10 records by Average Rating (stars) and display their:
    - item_name attribute from Item entity
    - Average(stars) as Average Rating
    - Count(item_id) - that is the number of ItemRatings entities associated to each Item with the same ItemName.
    - These should be sorted by Average Rating descending, then by item_name ascending.

```
SELECT
        I.item_name AS ItemName,
        ROUND(AVG(ir.stars),1) AS AverageRating,
        COUNT(ir.item_id) AS RatingCount
FROM Item i
INNER JOIN ItemRating ir
        ON i.item_id = ir.item_id
GROUP BY
        I.item_name
ORDER BY
        AverageRating DESC,
        I.item_name ASC
LIMIT 10;
```

Auction Statistics

Abstract Code

- Read from Auction, Item,ItemRating entities.
- Aggregate information about auctions:
  - Total number of active auctions (auction_end_time is past current time)
    SELECT COUNT(*) total_active_auctions from Auction WHERE NOW() < scheduled_end_time AND canceled_time IS NULL

  - Total number of finished auctions
    - current time is past auction_end_time AND
    - auction was not canceled

      SELECT COUNT(*) total_finished_auctions FROM Auction WHERE NOW() > scheduled_end_time AND canceled_time IS NULL

  - Total number of auctions with winners (either via Get It Now or auction)

    SELECT COUNT(*) winning_auctions FROM Auction WHERE canceled_time IS NULL AND winner IS NOT NULL

  - Total number of canceled auctions

    SELECT COUNT(*) total_canceled_auctions FROM Auction WHERE canceled_time IS NOT NULL
- Aggregate information about ratings:
  - Total number of items with ratings

    SELECT COUNT(*) items_rated FROM Item INNER JOIN ItemRating ir ON item_ID=ir.item_ID

  - Total number of items without ratings

```
SELECT COUNT(*) items_not_rated FROM Item WHERE item_ID NOT IN (
SELECT item_ID FROM Item INNER JOIN ItemRating ir ON item_ID=ir.item_ID)
```

```
SELECT * FROM
(SELECT COUNT(*) total_active_auctions from Auction WHERE NOW() < scheduled_end_time
AND canceled_time IS NULL) AS total_active_auctions ,
(SELECT COUNT(*) total_finished_auctions FROM Auction WHERE NOW() > scheduled_end_time
AND canceled_time IS NULL) AS total_finished_auctions,
(SELECT COUNT(*) winning_auctions FROM Auction WHERE canceled_time IS NULL AND winner
IS NOT NULL) AS winning_auctions,
(SELECT COUNT(*) total_canceled_auctions FROM Auction WHERE canceled_time IS NOT NULL)
AS  total_canceled_auctions,
(SELECT COUNT(*) items_rated FROM Item i INNER JOIN ItemRating ir ON i.item_id=ir.item_id)
AS  items_rated,
(SELECT COUNT(*) items_not_rated FROM Item WHERE item_id NOT IN ( SELECT i.item_id
FROM Item i INNER JOIN ItemRating ir ON i.item_id=ir.item_id)) AS items_not_rated;
```

- Return report with auction and ratings report details.

Canceled Auction Details

Abstract Code

- User clicked on ***Cancelled Auction Details*** button from **Main Menu**.
- Run the Cancelled Auction Details task; query the information about auction to get an aggregate report based on the cancellation.

```
SELECT
    i.item_id,
    i.listed_by,
    a.canceled_time,
    a.cancelation_reason
FROM Auction a
INNER JOIN Item i ON a.item_ID = i.item_ID
WHERE a.canceled_time IS NOT NULL
ORDER BY i.item_id DESC
```